AWS Architecting Cheat Sheet

Based on "Architecting on AWS" (v6.8) Official Course Curriculum

A condensed, student-friendly reference covering every module of the Architecting on AWS course: core services, key architectural concepts, and exam/interview-style tips. Use it for quick review, not as a replacement for the full course or hands-on labs.

> **Source**: Synthesized from the official AAI "Architecting on AWS 6.8" training deck (Modules 01–15 plus Appendix; the purely administrative Module 00 "Welcome" is omitted — no technical content). Content reflects what is taught in the course as of the 6.8 revision; always cross-check against current AWS documentation, since services and features evolve.

Table of Contents

#	Module
01	Introduction
02	The Simplest Architectures
03	Adding a Compute Layer
04	Adding a Database Layer
05	Networking in AWS Part 1
06	Networking in AWS Part 2
07	AWS Identity and Access Management (IAM)
08	Elasticity, High Availability, and Monitoring
09	Automation
10	Caching
11	Building Decoupled Architectures
12	Microservices and Serverless Architectures
13	RTO/RPO and Backup Recovery Setup
14	Optimizations and Review
15	Course Wrap Up
—	Appendix: Cloud Economics, Shared Responsibility, and Security Deep Dive

---

Module 01: Introduction

This module frames the "why" of cloud computing and AWS, tracing AWS's origin from Amazon.com's internal infrastructure challenges. It introduces the AWS Well-Architected Framework's five pillars and the global infrastructure model (Regions, Availability Zones, edge locations) that underpins all AWS architectures. It closes by previewing the large-scale reference architecture that the rest of the course builds toward.

Key Concepts

- Six advantages of cloud computing: trade capital expense for variable expense, benefit from economies of scale, stop guessing capacity, increase speed/agility, focus on business value (not infrastructure), go global in minutes.
- **AWS Well-Architected Framework** — five pillars: **Security**, **Reliability**, **Cost Optimization**, **Operational Excellence**, **Performance Efficiency**.
- Infrastructure hierarchy: **Region** → **Availability Zone** (AZ) → Data Center.
- An AZ consists of one or more physically separated data centers with independent power/network; a data center belongs to only one AZ.
- Regions contain **two or more AZs**; data is not replicated across regions automatically — replication is the customer's responsibility.
- Core applications are deployed **N+1** across data centers so failure of one doesn't cause an outage.
- **Edge locations**/Regional Edge Caches deliver content with lower latency (used by **Amazon CloudFront** and **Amazon Route 53**).
- **Mechanical sympathy** — choose the technology approach that matches the access pattern/workload (performance efficiency pillar).

AWS Services Covered

- **AWS Well-Architected Tool** — self-service review of workloads against AWS best practices across the five pillars.
- **Amazon CloudFront** — content delivery network using global edge locations/regional edge caches for low-latency delivery.
- **Amazon Route 53** — DNS service supported by AWS edge locations (mentioned as part of global infrastructure).
- **AWS GovCloud (US)** — isolated region for US government workloads with specific regulatory/compliance needs.
- **AWS Local Zones** — infrastructure deployments that place compute, storage, and select services closer to large population centers for single-digit-millisecond latency to end users. *(editor's note)*

Exam/Interview Tips

- Know the hierarchy: a Region has **≥2 AZs**; an AZ has **≥1 data center**; a data center belongs to exactly one AZ.
- AWS never moves/replicates your data out of a region automatically — cross-region replication must be explicitly configured by you.
- Memorize the five **Well-Architected pillars**: Security, Reliability, Cost Optimization, Operational Excellence, Performance Efficiency.

- The **six cloud advantages** are a common exam theme — be able to name all six, not just recite "cost savings."

Module 02: The Simplest Architectures

This module introduces Amazon S3 as the foundation for simple, durable cloud architectures, covering object storage fundamentals, access control mechanisms, storage classes/lifecycle management, and data transfer options. It also covers Amazon S3 Glacier for low-cost archival and closes with the business criteria for choosing an AWS Region. The accompanying lab has students host a static website on S3.

Key Concepts

- **S3** is object-level storage: a full object must be re-uploaded to change any part of it (no in-place byte edits).
- Durability vs. availability: **S3** is designed for **99.999999999%** (11 nines) durability via redundant storage across multiple facilities/devices.
- Default **S3** access is private; public access requires explicitly disabling **"Block Public Access"** settings.
- Layered access control: resource policy (owner-only by default) → **ACLs** → **bucket policies** → access points; IAM covered separately in Module 07.
- **Versioning** protects against accidental overwrite/delete (delete marker instead of permanent deletion); **S3 Object Lock** adds WORM-style retention/legal holds (requires versioning).
- **CORS** allows client-side web apps in one domain to access S3 resources in another domain.
- **Multipart upload** improves throughput, enables pause/resume, and supports objects up to **5 TB**; required above certain sizes.
- **S3 Transfer Acceleration** uses CloudFront edge locations to speed up uploads over long distances.
- Storage class selection is a cost/access-frequency tradeoff; **lifecycle policies** automate moving/deleting objects by age to reduce cost.
- S3 good fit: write-once-read-many, spiky access, large/diverse user content, growing datasets. Poor fit: block storage needs, frequently changing data, long-term archival (use Glacier instead).
- S3 billing: pay for storage (GB/month), requests (PUT/COPY/POST/LIST/GET), and data transfer OUT; transfer IN and transfer to same-region EC2/CloudFront are free.
- **Glacier** retrieval tiers trade speed for cost: **Expedited** (1–5 min), **Standard** (3–5 hrs), **Bulk** (5–12 hrs, cheapest).
- Region selection criteria: data residency/compliance laws, latency/proximity to users, service availability per region, and cost differences.
- Large-scale data migration tools scale by volume: console/CLI/API (small/in-network data) → **AWS DataSync** (ongoing automated transfer) → **AWS Transfer Family** (SFTP workflows) → **AWS Snowball** (petabyte-scale physical transport) → **AWS Snowmobile** (exabyte-scale).

AWS Services Covered

- **Amazon S3** — scalable object storage; durable, used for static content hosting, backups, data lakes, analytics source data.
- **Amazon S3 Glacier** — very low-cost, long-term archival storage with retrieval times from minutes to hours.
- **Amazon S3 Standard-Infrequent Access** (S3 Standard-IA) — lower-cost tier for data needing immediate access but accessed infrequently.

- **Amazon S3 One Zone-IA** — infrequent-access storage in a single AZ, ~20% cheaper than Standard-IA, lower resilience.
- **Amazon S3 Intelligent-Tiering** — automatically moves objects between access tiers based on changing access patterns.
- **Amazon S3 Glacier Deep Archive** — cheapest storage tier for data rarely/never accessed, retrieval within ~12 hours.
- **Amazon S3 Reduced Redundancy Storage** (RRS) — lower-redundancy storage for noncritical, reproducible data.
- **AWS DataSync** — automated data transfer service between on-premises storage and Amazon S3/Amazon EFS, faster than open-source tools.
- **AWS Transfer Family** — fully managed SFTP/FTPS/FTP service for transferring files directly into Amazon S3.
- **AWS Snowball** — petabyte-scale physical data transport appliance, no code/hardware purchase required.
- **AWS Snowball Edge** (Optimized) — Snowball variant with onboard compute for edge processing in disconnected environments.
- **AWS Snowmobile** — exabyte-scale physical data transport via a shipping-container-sized truck trailer.
- **Amazon CloudFront** — CDN; used as an S3 origin and to accelerate uploads via S3 Transfer Acceleration.
- **Amazon SNS** (Simple Notification Service) — used to track/notify on Snowball job status.
- **AWS KMS** (Key Management Service) — manages encryption keys protecting data in transit via Snowmobile.
- **AWS Policy Generator** — tool to help author S3 bucket policies (mentioned in access control context).
- **Amazon QuickSight** — BI tool example for analyzing exported S3 storage class analytics.

Exam/Interview Tips

- Know the 4 **"Block Public Access"** settings are ON by default on new buckets — public access must be deliberately disabled.
- Distinguish **ACLs** (grantee-based XML, per object/bucket) vs. **bucket policies** (JSON, up to 20 KB, can grant cross-account access) vs. IAM policies.
- Max single S3 object size is **5 TB**; console upload is capped at **160 GB** — use CLI/API/multipart for larger files.
- Glacier retrieval is never instantaneous — minimum is minutes (**Expedited**), not seconds; this is a common trick question.
- Region choice criteria to recall: compliance/data residency, latency to users, service/feature availability, and cost — in roughly that priority order per the course.

Module 03: Adding a Compute Layer

This module adds Amazon EC2 and its surrounding ecosystem to the architecture: how instances are launched (AMIs, user data, instance metadata), how to attach storage (Amazon EBS, instance store, shared file systems), how to pick the right instance family/size, and how to control cost and placement through pricing models, tenancy options, and placement groups.

Key Concepts

- **EC2** instances are cloud equivalents of physical/virtual servers — usable for web hosting, databases, authentication, or any general server workload.
- Disposable/ephemeral compute mindset: instances can be created and destroyed in seconds, enabling fast iteration and reduced risk from mistakes.
- **AMIs** provide repeatability, reusability, recoverability, and backup — launch multiple identical instances from one AMI.
- Three AMI sources: AWS-provided prebuilt AMIs, **AWS Marketplace**, or self-created ("golden image") AMIs; community AMIs exist but are unvetted and risky for production.
- **User data** scripts automate post-launch configuration (patches, software installs); runs once at first boot with root/admin privilege.
- **Instance Metadata Service** (link-local address **169.254.169.254**) lets a running instance query its own instance-id, IP, hostname, mac address, etc.
- **Instance store** is ephemeral (lost on stop/termination); **Amazon EBS** is persistent, durable block storage that survives shutdown and can be backed up as a snapshot/AMI.
- An EBS volume attaches to only one instance at a time; multiple EBS volumes can attach to one instance.
- SSD-backed EBS volumes suit IOPS-heavy transactional workloads; HDD-backed volumes suit throughput-heavy, large sequential workloads and cannot be boot volumes.
- Shared storage across multiple instances requires a file system (**Amazon EFS**/**Amazon FSx**), not EBS (single-attach) or S3 (object, not block/file semantics for frequent small changes).
- EC2 instance naming convention: family letter (e.g., M) + generation number (e.g., 5) + size (e.g., large) → "m5.large"; sizes scale by coefficient (xlarge, 2xlarge...).
- Instance family choice is a cost/performance decision based on workload: **General Purpose** (burstable, balanced), **Compute Optimized**, **Memory Optimized**, **Accelerated Computing** (GPU), **Storage Optimized**.
- Newer-generation instances typically offer better price-to-performance than older generations even at a higher hourly rate.
- Pricing model choice depends on workload predictability/flexibility: **On-Demand** (no commitment), **Reserved Instances** (pre-pay for discount), **Savings Plans** (flexible commitment-based discount), **Spot Instances** (steep discount, interruptible, needs fault tolerance).
- **Spot Instances** receive a **2-minute** interruption notice before reclaim — only suitable for stateless/fault-tolerant, flexible workloads.
- Tenancy options trade cost for isolation/control: **Default** (shared hardware) < **Dedicated Instance** (isolated, non-specific hardware) < **Dedicated Host** (specific physical server, supports BYOL licensing).

- Tenancy cannot be changed between Default and Dedicated/Host after launch; Dedicated Instance ↔ Dedicated Host is changeable.
- **Placement groups** address specific networking/resilience needs: **Cluster** (lowest latency, same AZ), **Spread** (isolate critical instances across distinct hardware, up to **7 per AZ**), **Partition** (large distributed systems like HDFS/Cassandra, isolates failure to a partition).
- Resource tagging (key/value metadata) supports management, search, filtering, cost tracking, and access control; favor consistent, standardized, plentiful tags.

AWS Services Covered

- **Amazon EC2** (Elastic Compute Cloud) — resizable virtual server instances for general compute workloads.
- **Amazon Machine Image** (AMI) — template (root volume, launch permissions, block device mapping) used to launch EC2 instances.
- **EC2 Image Builder** — automates creation, testing, and distribution of golden AMIs.
- **Amazon EBS** (Elastic Block Store) — persistent, durable block storage volumes attachable to a single EC2 instance; supports **General Purpose SSD**, **Provisioned IOPS SSD**, **Throughput Optimized HDD**, **Cold HDD** types.
- **EC2 Instance Store** — temporary block storage physically attached to the host, lost on stop/termination; for caches/buffers/scratch data.
- **Amazon EFS** (Elastic File System) — managed, scalable NFSv4 shared file system for Linux workloads, mountable by many EC2 instances across AZs/VPCs/accounts.
- **Amazon FSx for Windows File Server** — managed native Windows (SMB/NTFS) shared file system with Active Directory integration.
- **Amazon FSx for Lustre** — managed, high-performance POSIX file system for HPC/ML/media processing, integrates with Amazon S3.
- **AWS License Manager** — manages and enforces software license usage rules across EC2 instances.
- **AWS Compute Optimizer** — ML-based recommendations for optimal EC2 instance types to cut cost and improve performance.
- **Elastic Fabric Adapter** (EFA) — high-performance network adapter for HPC workloads requiring low-latency inter-instance communication.
- **AWS Direct Connect** — dedicated network connection used for on-premises access to Amazon EFS/FSx.
- **AWS VPN** — VPN connectivity option for on-premises access to Amazon EFS/FSx.
- **Amazon CloudWatch** (Events/EventBridge) — delivers Spot Instance interruption notices.

Exam/Interview Tips

- Remember **EBS** = single-instance attach, block storage, persistent; **EFS/FSx** = multi-instance attach, shared file system — a frequent exam distinction.
- Instance store is ephemeral and wiped on stop/terminate — never use it for data that must persist.
- **Spot Instances** get exactly a **2-minute** warning before termination; design for checkpointing if using Spot.
- Standard **Reserved Instances** give the biggest discount (~75%) but least flexibility; Convertible RIs (~54%) allow attribute changes; **Savings Plans** trade specificity for flexibility across instance families/regions.

- **Dedicated Host** vs. **Dedicated Instance**: Host = specific physical server (needed for per-socket/core BYOL licensing); Instance = isolated but non-specific hardware. Tenancy can't flip to/from Default after launch.
- **Cluster** placement group = lowest latency within one AZ; **Spread** = isolate critical instances across hardware (max **7/AZ**); **Partition** = large distributed data systems (HDFS, Cassandra, HBase).

Module 04: Adding a Database Layer

Overview

This module covers how to choose and secure a dedicated database layer that is separate from the application tier, highly available, and easy to scale. It contrasts relational and non-relational database models, compares unmanaged (self-hosted/EC2) vs. managed (AWS) database operating responsibility, and introduces Amazon RDS and Amazon DynamoDB as the primary focus services. It closes with security controls for both services and tools for migrating existing databases into AWS.

Key Concepts

- Database selection criteria: scalability, total storage requirements, object size/type, and durability/availability/recoverability needs (including regulatory obligations).
- Relational (SQL) databases: fixed schema, rows/columns, queried with **SQL**, scale vertically; best for strict schema enforcement and complex joins without extreme read/write demands.
- Non-relational (NoSQL) databases: dynamic schema, key-value/document/graph storage, queried via document collections, scale horizontally; best for high-volume simple data, rapid scaling, no complex joins.
- Managed vs. unmanaged responsibility model: on-premises/EC2-hosted databases leave you responsible for OS/DB patching, backups, scaling, and HA; fully managed AWS database services take over everything except application-level optimization.
- **DynamoDB** data model: tables of items (schemaless), **partition key** (hash key) and optional **sort key** (range key), hash function distributes items across partitions, auto-partitioning as data/throughput grows.
- DynamoDB consistency model: **eventually consistent** reads (default, **0.5 RCU** per 4 KB) vs. **strongly consistent** reads (**1 RCU** per 4 KB).
- DynamoDB global tables: multi-region, multi-master replication without building custom replication.
- Database security layers (common to RDS/DynamoDB): access control, encryption at rest, encryption in transit, and monitoring/notifications.
- Migration strategy: one-time vs. ongoing (continuous replication) migration; use Snowball Edge when the database is too large or network too slow/insecure for online migration; schema conversion is separate from data migration.
- Choosing between RDS and DynamoDB often comes down to query pattern, not just data volume: if the application needs multi-table joins and transactions, relational wins even at smaller scale *(editor's note)*.

AWS Services Covered

- **Amazon RDS** (Amazon Relational Database Service) – fully managed relational database service; provisions instances in minutes and scales vertically; supports **MySQL**, **MariaDB**, **PostgreSQL**, **Oracle**, **SQL Server** (**16TB** max storage for most engines).
- **Amazon Aurora** – MySQL- and PostgreSQL-compatible managed relational engine with up to **5x MySQL** / **3x PostgreSQL** throughput, replicates data six ways across three **Availability Zones**, **64TB** max storage, supports global databases.
- **Amazon DynamoDB** – fully managed non-relational (NoSQL) database with extreme horizontal scaling, single-digit millisecond latency, built for event-driven/serverless applications.

- **Amazon Neptune** – managed non-relational graph database option (mentioned as one of the broader database choices).
- **Amazon ElastiCache** – managed in-memory caching service listed among non-relational database options (Memcached shown in architecture diagrams).
- **Amazon Redshift** – managed relational data warehouse option for analytics workloads.
- **AWS Database Migration Service** (AWS DMS) – migrates databases to/from AWS (EC2, RDS, S3, on-premises), supports homogeneous and heterogeneous engine migrations, one-time or ongoing replication.
- **AWS Schema Conversion Tool** – standalone tool that converts database schema from one engine to another (e.g., Oracle → Aurora/MySQL/PostgreSQL; SQL Server → Aurora/MySQL/PostgreSQL).
- **AWS Snowball Edge** – physical data transfer appliance integrated with AWS DMS for migrating multi-terabyte/petabyte databases without consuming network bandwidth.
- **Amazon EC2** – used to host application servers and/or self-managed database instances in the lab architecture.
- **Amazon SNS** (Amazon Simple Notification Service) – delivers Amazon RDS event notifications (email/text) for events like failover, low storage, or security group changes.

Exam/Interview Tips

- Know the decision checklist cold: relational = fixed schema, joins, vertical scaling; non-relational = dynamic schema, horizontal scaling, high throughput.
- Remember DynamoDB's default read consistency is **"eventually consistent"** (cheaper, **0.5 RCU**); **"strongly consistent"** reads cost **1 RCU** and may be unavailable during network issues.
- AWS DMS does NOT perform schema/code conversion — pair it with the **AWS Schema Conversion Tool** for heterogeneous migrations.
- For AWS DMS, either the source or target must be in **Amazon EC2/RDS** — you cannot migrate directly between two on-premises databases.
- RDS encryption at rest covers storage, automated backups, read replicas, and snapshots together — you can't selectively encrypt just one of these.

Module 05: Networking in AWS Part 1

Overview

This module introduces Amazon VPC as the foundation for deploying isolated, networked AWS environments. It covers VPC structure and IP addressing, subnet design (public vs. private), routing, internet/NAT gateways, elastic network interfaces, and the core security constructs (security groups and network ACLs) used to protect resources.

Key Concepts

- A **VPC** is a logically isolated virtual network dedicated to your AWS account, deployed into one AWS Region, and can span multiple **Availability Zones**.
- Organizational patterns: single VPC (small teams, HPC, identity management), multi-VPC (single team/organization), multi-account (large/complex organizations) — choice driven by team size, compliance, and governance needs.
- VPC CIDR blocks define the private IP range (e.g., **/16 to /28**); subnets are subsets of the VPC CIDR, each pinned to one Availability Zone, and cannot overlap with each other.
- AWS reserves **5 IP addresses** in every subnet (network address, VPC router, DNS, future use, broadcast).
- Subnets should be organized by internet accessibility (public vs. private), not by application tier — this gives clean, subnet-level isolation.
- Public subnets route to an **internet gateway**; private subnets route outbound-only traffic through a **NAT gateway** and have no direct inbound internet access.
- Route tables direct traffic between VPC resources; every VPC has a default (main) route table, and best practice is to use custom route tables per subnet.
- Subnet sizing best practice: prefer larger subnets (**/24 or larger**) to simplify placement and avoid running out of IPs; private subnets typically need far more IPs than public subnets since most resources are private.
- Elastic network interfaces (ENIs) can carry private IP, Elastic IP, and MAC address when moved between instances in the same Availability Zone — useful for management networks, dual-homed instances, and low-budget HA failover.
- **Elastic IP** addresses are static public IPv4 addresses (**5 per Region** by default) that can be remapped instantly between instances/interfaces for failover.
- **Defense in depth**: combine **security groups** (stateful, instance-level) with **network ACLs** (stateless, subnet-level) so a misconfiguration in one layer doesn't fully expose resources.
- Security groups default to deny-all inbound / allow-all outbound; rules are **stateful** (return traffic automatically allowed).
- Network ACLs default to allow-all in both directions when first created on a VPC, but custom ACLs default to deny-all until rules are added; rules are **stateless** (must define both inbound and outbound explicitly).

AWS Services Covered

- **Amazon VPC** (Amazon Virtual Private Cloud) – isolated virtual network for your AWS resources; defines IP ranges, subnets, route tables, and gateways.

- **Amazon EC2** (Amazon Elastic Compute Cloud) – compute instances deployed into VPC subnets; associated with elastic network interfaces, security groups, and Elastic IP addresses.
- **Amazon RDS** (Amazon Relational Database Service) – managed database used in the lab's data subnet, protected by a dedicated DB security group.
- **Internet Gateway** – horizontally scaled, redundant VPC component enabling communication between VPC instances and the internet; required for public subnet connectivity.
- **NAT Gateway** – allows private subnet instances to initiate outbound internet traffic while blocking unsolicited inbound traffic.
- **Security Groups** – stateful virtual firewalls controlling inbound/outbound traffic at the instance (ENI) level.
- **Network ACLs** (Access Control Lists) – stateless firewalls controlling traffic at the subnet boundary.
- **Elastic Network Interfaces** – virtual NICs attachable/movable across EC2 instances within the same Availability Zone.
- **Elastic IP Addresses** – static public IPv4 addresses that can be remapped between instances for immediate failover.

Exam/Interview Tips

- Know the hard numbers: default VPC limit is **5 per Region** per account; Elastic IPs are capped at **5 per Region**; subnet CIDR range is **/16 to /28**.
- A classic exam trap: security groups are **stateful** (return traffic auto-allowed) while network ACLs are **stateless** (must allow both directions explicitly) — don't mix these up.
- Remember each subnet lives in exactly one Availability Zone and cannot span zones; an AZ can hold multiple subnets.
- Private subnets have no route to an internet gateway — they reach the internet only via a NAT gateway sitting in a public subnet.
- Best-practice subnet design is "public vs. private" based on internet accessibility, not "per application tier" — tiering should be handled with security groups instead.

Module 06: Networking in AWS Part 2

Overview

This module extends networking concepts to connecting VPCs and on-premises networks, accessing AWS services privately via VPC endpoints, distributing traffic with load balancers, and designing for high availability across Availability Zones and Regions using Amazon Route 53. It addresses scaling an application to a larger, variable user base while tolerating Availability Zone failures.

Key Concepts

- Hybrid connectivity options: VPN (over the internet, via a **Virtual Private Gateway** and Customer Gateway) vs. **AWS Direct Connect** (dedicated private 1/10 Gbps circuit) — DX gives predictable performance, lower data transfer costs, and supports a VPN as a backup path.
- VPC-to-VPC connectivity: **VPC Peering** is a non-transitive, one-to-one private connection (no gateway required, stays on AWS backbone) good for a small number of VPCs; **Transit Gateway** is a hub-and-spoke model that scales to thousands of VPCs/on-premises networks with centralized route tables, supporting full-mesh or isolated routing designs.
- General multi-VPC design principles: avoid overlapping CIDR blocks, connect only VPCs that must communicate, and design for scalability and high availability.
- **VPC Endpoints** let resources reach AWS services privately without an internet gateway, NAT, VPN, or Direct Connect; two types: **Gateway Endpoints** (S3, DynamoDB — route table target) and **Interface Endpoints** (AWS PrivateLink — ENI with private IP, used for many other services).
- **Elastic Load Balancing** (ELB) distributes traffic across EC2 instances, containers, or IPs; three types differ by OSI layer: **Application Load Balancer** (Layer 7, HTTP/HTTPS, content-based routing), **Network Load Balancer** (Layer 4, TCP/TLS/UDP, extreme performance, static IP per AZ), **Classic Load Balancer** (previous generation, both layers).
- High availability pattern: assume everything fails and design backward — use redundant instances/replicated standby databases instead of single points of failure.
- Start with at least **two Availability Zones** per Region; load balancer health checks detect and route around unhealthy instances/AZs automatically.
- **Deregistration delay** lets in-flight requests finish before an instance is removed from a load balancer (integrates with Auto Scaling for safe instance replacement).
- **Amazon Route 53** provides DNS, domain registration, health checks, and multiple traffic-routing policies for multi-region HA and failover; for the fastest possible failover beyond DNS TTL limits, use **AWS Global Accelerator**.
- Route 53 routing policies: simple (round robin), weighted, latency-based, geolocation, geoproximity (with traffic bias), multi-value answer, and health-check-based DNS failover.
- Direct Connect connections are not encrypted by default since they bypass the public internet; pair with a VPN over Direct Connect (or application-layer TLS) if data-in-transit encryption is required *(editor's note)*.

AWS Services Covered

- **AWS Direct Connect** – dedicated private 1 Gbps or 10 Gbps network connection between on-premises and AWS, bypassing the public internet.

- **Virtual Private Gateway** (VGW) – the AWS-side VPN concentrator used to establish IPsec VPN connections between a VPC and an external network.
- **AWS Transit Gateway** – fully managed hub that connects up to **5,000 VPCs** and on-premises networks through a single gateway with centralized routing.
- **VPC Peering** – private, non-transitive one-to-one connection between two VPCs using private IP addresses.
- **VPC Endpoints** (Gateway and Interface types) / **AWS PrivateLink** – private connectivity from a VPC to supported AWS services or partner services without traversing the internet.
- **Elastic Load Balancing** (ELB) – managed service distributing incoming traffic across EC2 instances, containers, and IP addresses; includes Application Load Balancer, Network Load Balancer, and Classic Load Balancer.
- **Amazon Route 53** – highly available, scalable DNS service with domain registration, health checks, and multiple routing policies.
- **AWS Global Accelerator** – improves failover speed and global traffic routing using static anycast IP addresses, recommended for stringent SLAs.
- **Amazon CloudWatch** – referenced as the destination for ELB/EC2 metrics used to trigger Auto Scaling actions.
- **Amazon EC2** – compute instances that sit behind load balancers and within peered/connected VPCs.
- **Amazon S3** (Amazon Simple Storage Service) – example service accessed through a Gateway VPC endpoint.
- **Amazon DynamoDB** – example service accessed through a Gateway VPC endpoint.

Exam/Interview Tips

- Distinguish **VPC Peering** (simple, non-transitive, limited scale) from **Transit Gateway** (hub-and-spoke, scales to thousands of VPCs, supports isolation via separate route tables) — a common scenario question.
- **Gateway endpoints** only support Amazon S3 and Amazon DynamoDB; everything else needing private connectivity uses **Interface Endpoints** (AWS PrivateLink).
- Match load balancer type to OSI layer: **ALB** = Layer 7 (HTTP/HTTPS, content routing), **NLB** = Layer 4 (TCP/TLS/UDP, static IP, extreme scale), **CLB** = legacy/both layers.
- "Start with two Availability Zones" is the default HA guidance for tolerating a single AZ failure; additional AZs further reduce blast radius for very high-availability requirements.
- Route 53 health-check failover has DNS caching/TTL limits — know that **Global Accelerator** exists for near-instant failover when DNS propagation delay is unacceptable.

Module 07: AWS Identity and Access Management (IAM)

Overview

This module covers how to control who and what can access AWS resources as an organization grows and roles specialize. It explains the AWS account root user, IAM users/groups/roles/policies, federation options (STS, SAML, Amazon Cognito), and multi-account governance using AWS Organizations.

Key Concepts

- The **AWS account root user** has unrestricted, unlimited access and cannot have its privileges restricted — best practice is to secure its credentials and create an IAM admin user for daily use instead.
- **Least privilege** principle: grant only the permissions needed for a role/task; IAM users have no default permissions and must be explicitly granted access to the console, CLI, or SDK.
- IAM principals: IAM user, IAM role, identity provider (IdP), and federated user — any entity that can take action on an AWS resource.
- Policy evaluation logic: **explicit Deny** > explicit Allow > implicit Deny (default) — an explicit deny always wins.
- Policy types: **Identity-based policies** (attached to users/groups/roles; AWS-managed, customer-managed, or inline) vs. **Resource-based policies** (attached to resources like S3/KMS; always inline, no AWS-managed versions).
- IAM policies only govern access to AWS services/API calls — they have no visibility above the hypervisor (i.e., no control over OS-level access).
- **IAM Roles** decouple permissions from a specific user/group: permissions attach to the role, and users/services temporarily assume the role to get temporary security credentials — avoids distributing long-term credentials.
- Common role use cases: granting AWS resources (e.g., EC2) access to other AWS services, externally authenticated/federated users, cross-account access, and third-party access without sharing credentials.
- Federation patterns: **STS Identity Broker** (custom app queries AWS STS for temporary credentials after authenticating against a corporate directory) and **SAML federation** (enterprise IdP issues a SAML assertion, exchanged via AssumeRoleWithSAML for console access) — both avoid creating IAM users for every employee.
- **Amazon Cognito** separates **user pools** (sign-up/sign-in directory, supports social/SAML/OIDC login) from **identity pools** (exchange tokens for temporary AWS credentials to access AWS services) — used for web/mobile app authentication.
- Multi-account strategy: choose single account (centralized management), three accounts (dev/test/prod separation), or multiple accounts (autonomous departments) based on governance needs; cross-account access is not enabled by default.
- **AWS Organizations** centralizes management of multiple accounts: **Organizational Units** (OUs), **Service Control Policies** (SCPs) that bound what IAM policies can allow, consolidated billing, and automated account creation via API.
- **AWS Landing Zone** automates setup of a secure multi-account baseline (multi-account structure, account vending, user access, notifications).

- MFA (multi-factor authentication) should be enabled on the root user and on any IAM user with console access to privileged actions — it's a frequently tested baseline security control beyond policy design alone *(editor's note)*.

AWS Services Covered

- **AWS IAM** (AWS Identity and Access Management) – manages users, groups, roles, and policies to control access to AWS resources and APIs.
- **AWS STS** (AWS Security Token Service) – issues temporary, limited-privilege security credentials for IAM users, federated users, or assumed roles.
- **Amazon Cognito** – fully managed authentication, authorization, and user management for web/mobile apps via user pools and identity pools.
- **AWS Organizations** – centrally manages policies, billing, and account creation across multiple AWS accounts using OUs and Service Control Policies.
- **AWS Landing Zone** – automates setup of a secure, multi-account AWS environment following best practices (multi-account structure, account vending machine, notifications, user access).
- **AWS CloudTrail** – logs authenticated API requests to IAM and STS, used to trace AssumeRole/federated calls back to the originating user.
- **Amazon EC2** – example service referenced for instance-level IAM roles and STS-based protected actions.
- **Amazon S3** (Amazon Simple Storage Service) – example resource protected by resource-based (bucket) policies.
- **AWS KMS** (AWS Key Management Service) – example resource type that uses resource-based policies.

Exam/Interview Tips

- First instinct on any "root user used in an app/production" scenario question: stop using the root user, move to an **IAM role** (ideally with federation/web identity).
- Remember the policy evaluation order: **explicit deny** beats explicit allow beats implicit deny (the default when nothing matches).
- If a user can't access an S3 bucket, check both the **identity-based policy** (user/group/role) and the **resource-based policy** (bucket policy) — either can block access.
- Roles use temporary credentials and must be "assumed" (via console, CLI, AssumeRole API, or STS) — this is the standard answer for "grant temporary access" questions.
- Know the SCP relationship: a **Service Control Policy** in AWS Organizations sets the ceiling on what IAM policies can grant — if the SCP doesn't allow a service, no IAM policy within the account can override that.

Module 08: Elasticity, High Availability, and Monitoring

Overview

This module covers the three pillars of availability (fault tolerance, recoverability, scalability) and how AWS lets infrastructure expand and contract to match demand. It introduces monitoring as the foundation for reactive, elastic architectures, then walks through Auto Scaling strategies for compute and the various ways to scale relational and NoSQL databases.

Key Concepts

- **High availability** = **fault tolerance** (redundancy, no single point of failure) + **recoverability** (restoring service after disaster) + **scalability** (handling growth without redesign).
- **Elasticity**: infrastructure that intelligently expands/contracts with demand, unlike fixed-capacity traditional data centers.
- Three types of elasticity: **Time-based** (turn off unused resources, e.g. dev/test at night), **Volume-based** (match scale to current demand intensity), **Predictive-based** (scale ahead using known traffic trends/ML).
- **Monitoring** is the first step toward reactive, cost-saving, elastic architecture — covers resource utilization, security auditing, operational health, and application performance.
- **Auto Scaling** ways: **Scheduled** (predictable workloads), **Dynamic** (target tracking, e.g. CPU-based), **Predictive** (machine-learning based, least manual effort).
- **Auto Scaling groups** support mixed purchasing options (**On-Demand**, **Spot**, **Reserved**) in one group to optimize cost; configurable via Spot allocation strategy, On-Demand base/percentage.
- Set sensible min/max/desired capacity; scale out fast, scale in slowly; use **lifecycle hooks** for custom actions on launch/terminate; instances take minutes to become fully usable.
- Database scaling approaches: **read replicas** (horizontal, read-heavy offload), vertical push-button scaling (resize instance class, brief downtime except Aurora Serverless), **Aurora replicas** (up to **15**, read-only, multi-AZ), **Aurora Serverless** (auto start/stop/scale by **ACUs**, good for spiky/unpredictable workloads), **database sharding** (horizontal write scaling by splitting data into partitions/shards).
- **DynamoDB** scales via **Auto Scaling** (target utilization with bounds) or **On-Demand** (pay-per-request, no capacity planning, good for spiky/unpredictable traffic).
- **DynamoDB Adaptive Capacity** automatically boosts throughput for "hot" partitions but does not fix a poorly chosen (non-uniform) partition key — key design still matters.

AWS Services Covered

- **Amazon CloudWatch** — Collects metrics, logs, and events; triggers alarms and automated actions (e.g., scaling, notifications) based on resource state.
- **AWS CloudTrail** — Records all API calls/account activity for auditing, security analysis, and compliance.
- **Amazon VPC Flow Logs** — Captures accepted/rejected/all IP traffic metadata for a VPC, subnet, or ENI for troubleshooting and security monitoring.
- **AWS Cost Explorer** — Visualizes and forecasts AWS spending patterns over up to 13 months of historical data.
- **Amazon EC2 Auto Scaling** — Automatically launches/terminates EC2 instances across Availability Zones based on demand or schedule; integrates with load balancers.

- **Amazon RDS** — Managed relational database supporting read replicas, vertical scaling, and storage Auto Scaling.
- **Amazon Aurora** — MySQL/PostgreSQL-compatible relational database with up to 15 low-latency read replicas per cluster.
- **Amazon Aurora Serverless** — On-demand auto-scaling Aurora configuration that starts, stops, and scales capacity (ACUs) based on application need.
- **Amazon DynamoDB** — Managed NoSQL database with Auto Scaling or On-Demand capacity modes and built-in Adaptive Capacity for hot partitions.
- **Amazon SNS** — Used here to deliver notifications from CloudWatch alarms (e.g., alerting on threshold breaches).
- **Amazon Route 53** — DNS service referenced as part of the end-to-end elastic web architecture.
- **Application Load Balancer** — Distributes incoming traffic across Auto Scaling group instances across Availability Zones.
- **Amazon CloudFront** — Referenced as the edge/CDN layer in the full elastic architecture diagram.

Exam/Interview Tips

- Know the difference: **fault tolerance** (redundancy) vs. **recoverability** (restoring after disaster) vs. **scalability** (handling growth) — these jointly define availability.
- **CloudWatch alarms** only trigger on sustained state changes, not momentary blips.
- Standard **RDS** scaling has brief downtime when changing instance class; **Aurora Serverless** scales with no downtime; storage scaling has zero downtime.
- **DynamoDB Adaptive Capacity** helps with uneven traffic but does NOT solve a bad partition key design — you still need high-cardinality, evenly accessed keys.
- You can switch **DynamoDB** from **provisioned** to **on-demand** once per day, but on-demand to provisioned as often as needed.
- CloudWatch retains metric data at different resolutions over time — standard 1-minute granularity is kept for **15 days**, which is a common source of confusion when troubleshooting older incidents. *(editor's note)*

Module 09: Automation

Overview

This module makes the case for automating infrastructure and deployments instead of manually building environments, since manual processes don't scale, lack audit trails/version control, and introduce risk. It covers Infrastructure as Code with AWS CloudFormation, fleet/configuration management with AWS Systems Manager and AWS OpsWorks, and fully managed application deployment with AWS Elastic Beanstalk.

Key Concepts

- Manual infrastructure changes are "risk without reward": no version control, no audit trail, inconsistent, doesn't scale.
- **Infrastructure as Code** (**IaC**): describe architecture in a template, treat it as source code (version-controlled, reviewed, repeatable, reusable).
- **AWS CloudFormation** turns a JSON/YAML template into a "**stack**" (collection of resources); supports **drift detection** to find manual changes that deviate from the template.
- CloudFormation **Conditions** let one template produce environment-specific variations (e.g., dev vs. production) while keeping configurations consistent.
- **Change sets** let you preview stack update impact before executing; **DeletionPolicy** preserves/backs up resources on stack delete/update.
- **AWS Quick Starts** are prebuilt, best-practice CloudFormation templates for standardized, fast (often one-click, <1 hour) reference deployments.
- Fleet-wide configuration management (patching, state enforcement, remote command execution) is handled by **AWS Systems Manager** rather than manual SSH/login.
- **AWS OpsWorks** provides configuration management (Chef/Puppet-based) with a **Stacks → Layers** model; layers can be OpsWorks layers (EC2 instances) or service layers (e.g., Amazon RDS); custom cookbooks/recipes customize layer behavior; lifecycle events (Setup, Configure, Deploy, Undeploy, Shutdown) hook into automation.
- **AWS Elastic Beanstalk** abstracts away infrastructure entirely — you supply code, it provisions and manages the stack (load balancer, Auto Scaling, etc.) while remaining transparent/inspectable.
- Control vs. convenience trade-off spectrum: **Amazon EC2** (do-it-yourself, most control) → **AWS CloudFormation** → **AWS OpsWorks** → **AWS Elastic Beanstalk** (most convenience, least manual control).

AWS Services Covered

- **AWS CloudFormation** — Describes AWS infrastructure as JSON/YAML templates and provisions/updates it repeatably as "stacks."
- **AWS Systems Manager** — Automates configuration, patching, and management of EC2 and on-premises instances at scale (Run Command, Patch Manager, Maintenance Windows, State Manager, Session Manager, Inventory).
- **AWS OpsWorks (Stacks / for Chef Automate / for Puppet Enterprise)** — Configuration management service to define and automate application architecture using Chef or Puppet.
- **AWS Elastic Beanstalk** — Fully managed PaaS that provisions infrastructure and manages the application stack so developers only supply code.

- **Amazon EC2** — Baseline compute service representing the "do it yourself" end of the control/convenience spectrum.
- **AWS CodeCommit** — Mentioned as a version-control option for storing CloudFormation templates as code.
- **Amazon Route 53** — Used to map a custom domain name to an Elastic Beanstalk environment's generated domain.
- **Amazon SQS** — Mentioned as an example resource provisioned alongside infrastructure (external app communication) in layered CloudFormation/OpsWorks designs.

Exam/Interview Tips

- **CloudFormation** templates can be **JSON** or **YAML** — know that both are equally supported and functionally equivalent.
- **Change sets** = preview before you commit; **DeletionPolicy** = protect/backup a resource on stack delete/update — don't confuse the two.
- **Quick Starts** are built by AWS Solutions Architects/partners following best practices — good "gold standard" starting point, not a custom solution.
- Know the convenience/control spectrum order: **EC2** → **CloudFormation** → **OpsWorks** → **Elastic Beanstalk**, used to pick the right automation tool for a scenario.
- **Elastic Beanstalk** still lets you see and access everything provisioned underneath — it's not a black box, despite being hands-off.

Module 10: Caching

Overview

This module explains when and what to cache, then walks the "data journey" from edge to web tier to database, showing where caching fits at each layer. It covers CDN/edge caching with Amazon CloudFront, session management strategies, and database-tier caching with Amazon DynamoDB Accelerator (DAX) and Amazon ElastiCache, including caching strategies (write-through, lazy loading, TTL).

Key Concepts

- Good caching candidates: data that's slow/expensive to query, relatively static but frequently accessed, and tolerant of some staleness.
- Caching benefits: faster application response, reduced database load/latency, lower cost — biggest wins for read-heavy workloads.
- Caching layers: client-side (browser), server-side reverse proxy, edge/CDN, and database caching — each intercepts repeated requests closer to the user.
- **CDN** (**Amazon CloudFront**) serves cached content from edge locations nearest the user instead of round-tripping to the origin; supports static, dynamic (zero TTL), streaming, secure, and user-input content.
- Cache expiration/invalidation approaches: **TTL** (preferred, lets origin confirm via If-Modified-Since), **renaming objects** (forces refresh), **explicit invalidation** (last resort — slow and expensive).
- **CloudFront** + **AWS WAF** + **Amazon Route 53** + **Elastic Load Balancing** combine to mitigate **DDoS** by absorbing/filtering traffic at the edge and scaling automatically.
- Session management is a caching concern: **sticky sessions** (session affinity at the load balancer, cost-effective but fragile on node failure) vs. storing session state in a distributed cache/database (e.g., **DynamoDB**) for resilience and faster retrieval.
- Database caching strategies: **Write-through** (cache updated on every DB write — never stale, but double-write penalty and can leave cache under-populated after scaling), **Lazy loading** (cache populated only on cache miss — avoids unnecessary data, but has cache-miss penalty and staleness risk), **TTL** (combines both — expires entries after a set time to balance freshness and efficiency).
- **Memcached** vs. **Redis** trade-off: Memcached is simple, multi-threaded, horizontally scalable for storage, no persistence/failover; Redis supports advanced data types, sorting, pub/sub, Multi-AZ auto-failover, and backup/restore.

AWS Services Covered

- **Amazon CloudFront** — AWS's global CDN; caches and delivers content from edge locations, supports WebSocket, integrates with AWS Shield/AWS WAF for security.
- **Amazon S3** — Common origin store for static content served/cached through CloudFront.
- **AWS WAF** — Web application firewall that filters HTTP traffic by rules (IP, headers, body, URI) to mitigate attacks including DDoS.
- **Amazon Route 53** — DNS service that is part of the globally distributed, edge-scaled architecture for resilience.
- **Amazon DynamoDB Accelerator (DAX)** — Fully managed, DynamoDB-API-compatible in-memory cache delivering microsecond read latency for DynamoDB tables.

- **Amazon ElastiCache** — Managed in-memory data store (Redis or Memcached) that sits in front of a database to reduce latency and DB load.
- **Amazon DynamoDB** — Used as a fast, scalable store for session state in the examples given.
- **Elastic Load Balancing** — Supports sticky sessions (session affinity) and spreads traffic across the scaled fleet.
- **Amazon EC2** — Hosts application/web servers that sit behind caching layers.
- **Amazon RDS** — Origin database that caching layers (ElastiCache, DAX) are placed in front of to offload read traffic.

Exam/Interview Tips

- Know the 3 cache invalidation methods and their trade-offs: **TTL** (cheap, slight delay) > **renaming objects** (immediate, more effort) > **explicit invalidation** (last resort, expensive).
- **Write-through** keeps cache fresh but costs double writes; **lazy loading** is efficient on storage but risks stale data and a 3-trip cache-miss penalty; **TTL** is often used to combine both approaches' benefits.
- **Memcached**: multi-threaded, scales horizontally, no persistence/replication. **Redis**: richer data structures, Multi-AZ failover, backup/restore — pick based on need for HA/durability vs. raw throughput/simplicity.
- **DAX** is specifically for **DynamoDB** (API-compatible, no code changes); **ElastiCache** is the general-purpose in-memory cache for any application/database.
- Cache only static or tolerant-of-staleness data — don't cache unique, rapidly changing, or rarely accessed data.
- **ElastiCache for Redis** also supports **Global Datastore** for cross-Region replication, useful for disaster recovery scenarios on top of its usual caching role. *(editor's note)*

Module 11: Building Decoupled Architectures

Overview

This module addresses the problem of tightly coupled architectures, where failure or scaling in one tier cascades to others, and introduces loose coupling using managed messaging intermediaries. It covers Amazon SQS (queue-based decoupling), Amazon SNS (pub/sub notification), and Amazon MQ (managed message broker for migrating existing applications), along with guidance on choosing between them.

Key Concepts

- **Tightly coupled** architectures: components directly dependent on each other; failure or scaling of one tier breaks or complicates the whole system.
- **Loose coupling**: insert managed intermediaries (load balancers, queues, topics) between tiers so failure/scaling of one component doesn't directly affect others.
- Queues (**Amazon SQS**) enable asynchronous processing, independent scaling of producers/consumers, and recovery from failed processing steps since unprocessed messages remain queued.
- SQS queue types: **Standard** (at-least-once delivery, best-effort order, near-unlimited throughput) vs. **FIFO** (exactly-once, strict order, lower throughput — **300 msg/s**, or **3,000/s** batched).
- Key SQS features: **Dead-letter queue** (captures messages that repeatedly fail processing), **Visibility timeout** (hides an in-flight message from other consumers while it's being processed), **Long polling** (waits for a message rather than returning empty immediately, reduces cost of polling).
- SQS message lifecycle: producer sends → distributed redundantly → consumer picks up (visibility timeout starts) → consumer deletes after processing (not automatic).
- Messaging anti-patterns: selectively filtering/querying messages in a queue, and oversized messages (should offload large payloads to **Amazon S3** and pass a reference instead).
- **Amazon SNS** follows **pub/sub**: publishers send to a topic, SNS pushes to all subscribers (fan-out) — contrasts with SQS's pull/poll, point-to-point model.
- SNS vs. SQS: SNS = push, no persistence, one-to-many, pub/sub; SQS = poll, persistent until deleted, one-to-one (per message), send/receive.
- SNS supports encryption via **AWS KMS** and multiple subscriber protocol types (Email, HTTP/S, SMS, SQS, Lambda).
- **Amazon MQ** targets migrating existing applications that already use standard messaging protocols (JMS, NMS, AMQP, STOMP, MQTT, WebSocket) without rewriting code; SQS/SNS are preferred for cloud-native applications.
- **Amazon MQ** achieves **HA** via active/standby brokers in two Availability Zones with a replicated message store.

AWS Services Covered

- **Amazon Simple Queue Service (Amazon SQS)** — Fully managed message queue that buffers and decouples producers from consumers; Standard and FIFO queue types.
- **Amazon Simple Notification Service (Amazon SNS)** — Managed pub/sub notification service that pushes messages from topics to multiple subscriber types.
- **Amazon MQ** — Managed Apache ActiveMQ message broker for migrating existing messaging applications using standard protocols (JMS, AMQP, MQTT, etc.).

- **Amazon S3** — Recommended store for large message payloads, referenced by the message instead of embedding the data.
- **AWS Lambda** — A supported SNS/SQS subscriber/target for processing messages or events.
- **AWS KMS** — Provides encryption keys used to encrypt SNS topics and SQS message contents.
- **Amazon EC2** — Common producer/consumer compute tier interacting with SQS/SNS.
- **Amazon RDS** — Example downstream persistence layer protected by queue-based decoupling (e.g., order processing).
- **Amazon CloudWatch** — Can trigger/receive SNS notifications based on alarm state (e.g., Auto Scaling group changes).

Exam/Interview Tips

- Remember the core distinction: **SNS** = push/pub-sub/no persistence/one-to-many; **SQS** = poll/point-to-point/persistent/one-to-one.
- **FIFO queues** guarantee exact-once, ordered delivery but cap throughput (**300 msg/s**, or **3,000/s** batched); **Standard queues** trade strict ordering/no-duplicates for near-unlimited throughput.
- **Visibility timeout** prevents duplicate processing of an in-flight message — set it to cover your worst-case processing time.
- Use a **dead-letter queue** to capture repeatedly failing messages instead of losing them.
- Choose **Amazon MQ** for lift-and-shift of existing broker-based apps (protocol compatibility); choose **SQS**/**SNS** for new, cloud-native, highly scalable applications.
- SQS messages can be retained for up to **14 days** (configurable retention period), so a consumer outage doesn't necessarily mean lost messages as long as it recovers within that window. *(editor's note)*

Module 12: Microservices and Serverless Architectures

Overview

This module moves from decoupled monolith to true microservices, covering how to build independent, API-communicating services and run them efficiently using containers (Amazon ECS, AWS Fargate) or serverless compute (AWS Lambda, API Gateway, Step Functions). It contrasts containers vs. virtual machines, explains serverless benefits, and walks through a full example of converting a monolithic forum app and an order-processing workflow into serverless/microservice patterns.

Key Concepts

- **Microservices**: independent, autonomous, specialized services communicating over well-defined APIs instead of a tightly-coupled monolith.
- Each microservice can be developed, deployed, scaled, and updated independently without affecting others.
- **Containers** package code, dependencies, and config into a portable, lightweight, resource-isolated unit — no hypervisor required, much faster to start than VMs, smaller than VM images.
- Containers solve "runs differently in different environments" problems (dev/test/prod consistency).
- **Amazon ECS** launch types: **EC2 launch type** (you manage the cluster of instances) vs. **Fargate launch type** (no server/cluster management).
- Service discovery/observability for microservices: **AWS Cloud Map** (service discovery + health checks), **AWS App Mesh** (traffic routing, metrics/logs/traces via Envoy proxy).
- **Serverless** computing: build/run apps without provisioning, scaling, or managing servers; pay only when code runs.
- **Event-driven architecture**: Lambda functions triggered by events (S3 changes, DynamoDB streams, SNS, SQS, CloudWatch Events, ALB target groups, IoT Button, etc.).
- **Lambda Layers**: separate shared code/libraries/runtimes from function code; max **5 layers** per function; **250 MB** unzipped deployment limit.
- Orchestrating serverless workflows: **AWS Step Functions** models application logic as a **state machine** (sequential, parallel, retry, choice/if-else) using the JSON-based **Amazon States Language**.
- Decoupling order processing with a queue (e.g., introducing **Amazon SQS** between app and database) protects against lost transactions from deadlocks/race conditions/network issues.
- Serverless architecture benefits: focus on application code not infrastructure, pay only on invocation, naturally supports microservice design.
- A **dead-letter queue (DLQ)** can be attached to a Lambda function's asynchronous invocations, not just to SQS, to capture events that repeatedly fail processing. *(editor's note)*

AWS Services Covered

- **AWS Lambda** – Run stateless code without provisioning servers; triggered by events or schedules; supports Node.js, Java, Python, C#, Go, Ruby, PowerShell; max execution time 15 minutes; Lambda@Edge runs functions at CloudFront edge locations.
- **Amazon Elastic Container Service (Amazon ECS)** – Managed container orchestration service; schedules and scales Docker containers across a cluster; supports EC2 or Fargate launch types.
- **AWS Fargate** – Serverless compute engine for containers; runs ECS or EKS workloads without provisioning or managing clusters/servers.

- **Amazon Elastic Container Registry (Amazon ECR)** – Registry for storing and managing container images used by ECS/EKS.
- **Amazon Elastic Kubernetes Service (Amazon EKS)** – Managed Kubernetes service for running containerized applications (can also run on Fargate).
- **AWS Cloud Map** – Service discovery; registers application resources (DBs, queues, microservices) under custom namespaces and health-checks them.
- **AWS App Mesh** – Service mesh that provides traffic control, metrics, logs, and traces across microservices using the Envoy proxy.
- **Amazon API Gateway** – Managed "front door" for APIs; handles hundreds of thousands of concurrent calls; integrates with EC2, Lambda, or any web app; supports API keys, versioning/stages, SigV4 auth, VPC integration.
- **AWS Step Functions** – Serverless orchestration service; coordinates Lambda/microservices as visual state machines with retry, parallel, and choice logic.
- **Amazon S3** – Object storage; used for static website hosting and as an event source (e.g., triggering Lambda on file upload).
- **Amazon DynamoDB** – Managed NoSQL database; common target for Lambda-based data processing and as an event source via DynamoDB Streams.
- **Amazon SNS** – Pub/sub notification service; used to send alerts (e.g., low-inventory notifications, e-mail/SMS).
- **Amazon SQS** – Managed message queuing service; decouples components (e.g., customer order queue, dead-letter queue) to prevent data loss during processing failures.
- **Amazon CloudWatch Events** – Event bus/trigger source for scheduled or event-driven workflows (e.g., triggering Step Functions/Lambda).
- **Amazon Cognito** – User identity management/authentication for serverless apps (e.g., dashboard login, mobile backend identity).
- **AWS STS (Security Token Service)** – Issues temporary security credentials, used in serverless mobile backend patterns alongside Cognito.
- **Amazon ElastiCache** – In-memory caching service, referenced as part of a serverless mobile backend architecture.
- **Amazon RDS** – Managed relational database, referenced as a backend data store option in serverless architectures.
- **AWS WAF** – Web application firewall; shown protecting the API Gateway/serverless architecture from attacks.
- **Amazon CloudFront** – CDN used to serve static website assets and as an event source for Lambda@Edge.
- **AWS Elemental MediaConvert** – File-based video transcoding service used in the Video on Demand (VOD) reference architecture.
- **Amazon Route 53** – DNS service used for resolving traffic to the static website/API endpoints in the serverless reference architecture.

Exam/Interview Tips

- Know the **ECS launch-type** distinction cold: **EC2 launch type** = you manage servers/cluster; **Fargate** = no server management at all.
- Containers ≠ VMs: containers share the host OS kernel and skip the hypervisor, so they start in milliseconds and are much smaller than VM images.

- **Lambda** functions are capped at **15 minutes** of execution time — long-running jobs need a different compute option (e.g., ECS/Fargate, Step Functions orchestration).
- A Lambda function can use at most **5 layers**, and the total unzipped package (code + layers) is capped at **250 MB**.
- **Step Functions** uses the **Amazon States Language** (JSON) to define **state machines** with Task, Choice, Parallel, and Fail states — useful for retry/parallel/if-else logic that Lambda/API Gateway alone can't express.
- **SQS** between an app and its database is a classic fix for "lost order" failure modes caused by deadlocks, race conditions, or transient network errors.

Module 13: RTO/RPO and Backup Recovery Setup

Overview

This module covers disaster recovery (DR) planning on AWS: defining Recovery Point Objective (RPO) and Recovery Time Objective (RTO), surveying the AWS services relevant to backup/DR across storage, compute, networking, database, and automation, and walking through four standard DR strategies ordered by cost and recovery speed — Backup and Restore, Pilot Light, Fully Working Low-Capacity Standby, and Multi-Site Active-Active.

Key Concepts

- **Disaster recovery (DR)** = preparing for and recovering from any event that disrupts business continuity (hardware/software failure, outage, physical damage, human error).
- **High availability** (withstand component failure), **Backup** (protect data), and **Disaster recovery** (restore app/data after major disaster) are distinct but related availability concepts.
- **RPO (Recovery Point Objective)**: maximum acceptable amount of data loss, measured in time (how often must you back up?).
- **RTO (Recovery Time Objective)**: maximum acceptable downtime before service is restored (how long can you be unavailable?).
- RPO/RTO targets are driven by the financial impact of downtime/data loss, and drive the cost-effectiveness of the chosen DR solution.
- AWS regions are strictly isolated from each other, so a region can be used as a DR site for another region; an active-active multi-region setup lets you pull a misbehaving region out of DNS rotation.
- Four standard DR strategies, ordered from lowest to highest cost/fastest recovery: 1. **Backup and Restore** – cheapest, slowest; restore from backups (Amazon S3/Glacier) onto freshly launched infrastructure. 2. **Pilot Light** – core data replicated continuously; minimal/no compute running; rest of environment started up at disaster time. 3. **Fully Working Low-Capacity Standby (Warm Standby)** – scaled-down but fully functional environment always running; scaled up on disaster. 4. **Multi-Site Active-Active** – full-capacity environment running in AWS at all times alongside on-premises, sharing live production traffic.
- Trade-off spectrum: lower cost = higher RTO/RPO; higher cost (always-on duplicate) = lowest RTO/RPO.
- Best practices: start simple (backups first), incrementally improve RTO/RPO, check software licensing, run **"Game Day"** failure exercises, and test that backups/AMIs/snapshots actually work.

AWS Services Covered

- **Amazon S3** – Highly durable (11 nines) primary backup/DR storage target; supports versioning and cross-region replication (CRR) for automatic async copying between buckets in different regions.
- **Amazon S3 Glacier** – Low-cost archival storage for infrequent-access backups; data kept in regional vaults with a daily-updated inventory; CRR into Glacier possible only via S3 Standard.
- **Amazon EBS** – Block storage with point-in-time snapshots stored durably in S3; snapshots copyable across regions/accounts and encrypted in transit.
- **AWS Snowball** – Physical data transport appliance for moving >10TB in/out of AWS faster than internet transfer; useful for large-scale backup transfer or bulk retrieval.

- **AWS DataSync** – Securely and quickly (up to 10x faster than open-source tools) syncs on-premises or cloud file data into Amazon EFS over internet or Direct Connect.
- **AWS Backup** – Fully managed, centralized, policy- and tag-based backup service across EBS, EC2, RDS, DynamoDB, EFS, and Storage Gateway.
- **Amazon EC2** – Compute instances for recovery; supports automatic recovery on hardware failure (retains instance ID, IPs, EBS attachments) and custom/pre-built AMIs for fast relaunch.
- **Amazon Route 53** – DNS service with health checks, failover routing, and weighted routing between primary and DR sites.
- **Elastic Load Balancing (ELB)** – Distributes traffic across instances; can pre-allocate a load balancer with a known DNS name to simplify DR cutover.
- **Amazon VPC** – Used to extend on-premises network topology into the cloud for DR of internal/enterprise applications.
- **AWS Direct Connect (DX)** – Dedicated network connection from on-premises to AWS; lower cost, higher bandwidth, more consistent than internet-based DR replication traffic; a US DX circuit reaches all US regions including GovCloud (US).
- **Amazon RDS** – Managed relational database; supports cross-region snapshots, automated backups, and Read Replicas (combinable with Multi-AZ) for DR.
- **Amazon DynamoDB** – Managed NoSQL database; supports on-demand backups, point-in-time recovery (up to 35 days), and Global Tables for multi-region multi-master replication.
- **AWS CloudFormation** – Infrastructure-as-code service; templates allow fast, repeatable, automated redeployment of infrastructure during recovery.
- **AWS Elastic Beanstalk** – PaaS deployment service; redeploy an entire application stack in a few clicks, including rolling back to a previously uploaded version.
- **AWS OpsWorks** – Configuration management service (Chef/Puppet-based); automatic host replacement, used with CloudFormation to template and quickly re-provision environments to meet RTO.
- **AWS Storage Gateway** – Hybrid storage connecting on-premises environment to AWS storage; three interfaces — File Gateway (NFS/SMB to S3), Volume Gateway (iSCSI block storage, cached or stored, backed by EBS snapshots/S3), and Tape Gateway (virtual tape library backed by S3/Glacier).

Exam/Interview Tips

- Memorize the **RPO** vs. **RTO** distinction precisely: RPO = acceptable data loss (backup frequency), RTO = acceptable downtime (restore speed). Expect scenario questions asking you to pick the right term.
- Know the four DR strategies in cost/speed order: **Backup & Restore** < **Pilot Light** < **Warm Standby** (low-capacity) < **Multi-Site Active-Active** — cost and recovery speed increase together.
- **Pilot Light** keeps only data replication running (no/minimal compute); **Warm Standby** keeps a scaled-down but fully functional stack running 24/7 — don't confuse the two.
- **Storage Gateway** has exactly three gateway types — **File**, **Volume**, **Tape** — each gateway instance provides only one interface type.
- **Amazon S3 Glacier** cannot do Glacier-to-Glacier cross-region replication directly; you replicate via **S3 Standard CRR** into Glacier in the target region.
- DynamoDB point-in-time recovery covers up to **35 days** of continuous backups — worth recalling alongside the RPO/RTO discussion since it directly bounds your achievable RPO for a DynamoDB-backed workload. *(editor's note)*

Module 14: Optimizations and Review

Overview

This capstone module reviews the full course through a large reference architecture and a structured set of self-assessment questions covering resource fit, resiliency, decoupling, and scalability. It reinforces EC2 pricing model trade-offs (On-Demand, Reserved, Spot), the value of the AWS Well-Architected Framework, and closes with group exercises applying the course's concepts to new architecture problems.

Key Concepts

- Four review questions to evaluate any architecture: 1. Is it using the right resources (right-sized instance types, **managed services** vs. self-managed)? 2. Is it **resilient** (no single points of failure, can it self-heal and recover from disaster)? 3. Are components **loosely coupled** (can dependencies between pieces be removed, e.g., via microservices)? 4. Can it **scale** effectively from small to very large user loads (minimum infrastructure + Auto Scaling)?
- Managed services (e.g., **Amazon RDS**) remove undifferentiated heavy lifting so teams can focus on design/creative work instead of installation and operations.
- **"Replace, then investigate"** philosophy: when something fails, swap in a new resource first and do root-cause/postmortem after, rather than fixing in place.
- Loose coupling pattern: insert a queue (e.g., **Amazon SQS** with a **dead-letter queue**) between the application and database to prevent lost transactions during spikes or failures.
- Right-sizing takes time (roughly **3 months** of usage data) to determine actual required capacity; start bigger, then scale down based on observed utilization.
- Blending EC2 pricing models (**On-Demand** for baseline, **Spot** for supplemental/interruptible capacity, **Reserved** for steady-state) can reduce cost but Spot exposes workloads to interruption — only use Spot where sudden termination is acceptable.
- Since **2017**, **Spot Instances** no longer require bidding — you pay the current Spot price; diversify across instance families/sizes/AZs/regions (separate Spot pools) to reduce interruption impact; **Spot Fleets**/RequestSpotFleet can launch at scale, and instances can be configured to stop/hibernate instead of terminate on interruption.
- Resiliency is built incrementally: single AZ → multiple AZs with **Auto Scaling** → multiple regions, each step reducing single points of failure.
- The **AWS Well-Architected Framework**/Tool provides a standard way to review workloads against AWS best practices.
- 10 best practices for building systems on AWS: enable scalability, automate your environment, use disposable resources, loosely couple components, design services not servers, choose the right database solutions, avoid single points of failure, optimize for cost, use caching, secure infrastructure at every layer.

AWS Services Covered

- **Amazon EC2** – Core compute; discussion centers on choosing the right instance type and pricing model (On-Demand, Reserved, Spot).
- **Amazon RDS** – Managed relational database example used to illustrate the benefit of managed services over self-installed software.

- **AWS Auto Scaling** – Automatically adjusts EC2 capacity; Auto Scaling groups can span Availability Zones to improve resilience.
- **Elastic Load Balancing / Application Load Balancer** – Distributes traffic across instances/AZs in the resilient reference architecture.
- **Amazon SQS** – Queue used to decouple order processing and prevent data loss (customer order queue + dead-letter queue).
- **Amazon S3** – Storage tier for static assets in the large-scale reference architecture.
- **Amazon CloudFront** – CDN tier for static asset delivery in the reference architecture.
- **Amazon Route 53** – DNS layer in the reference architecture, including multi-region routing.
- **Amazon EFS** – Shared file storage mounted across AZs in the reference architecture.
- **AWS Well-Architected Tool** – Free console tool to review workloads against the AWS Well-Architected Framework best practices.

Exam/Interview Tips

- Be ready to recall the **"4 review questions"** (right resources / resiliency / loose coupling / scalability) — they summarize how to critique any proposed architecture.
- Know **Spot Instance** mechanics: post-**2017** pricing has no bidding, interruption impact is reduced by diversifying across pools (instance family/size/AZ/region) and by choosing stop/hibernate instead of terminate.
- Remember the **"replace first, investigate later"** operational philosophy — distinguishes cloud-native thinking from traditional on-prem troubleshooting.
- The **10 best-practices** list is a good checklist to recite in interviews when asked "how do you design for the cloud?"
- Spot Instances can offer savings of up to **90%** off On-Demand pricing, which is useful context for why blending pricing models matters for cost optimization. *(editor's note)*

Module 15: Course Wrap Up

Overview

A brief closing module pointing students to AWS Training and Certification resources and requesting course feedback. No new architectural content.

Key Concepts

- **AWS Certification** (aws.amazon.com/certification) validates AWS skills and knowledge.
- **AWS Training** (aws.amazon.com/training) builds skills to design, develop, deploy, and manage AWS applications.
- **Self-Paced Labs** (aws.amazon.com/training/self-paced-labs) offer hands-on practice with AWS technologies.
- Students are asked to submit course feedback via their aws.training transcript.

Appendix: Cloud Economics, Shared Responsibility, and Security Deep Dive

Overview

The Appendix revisits the six core advantages of cloud computing (expense, scale, capacity, speed, focus, global), then goes deeper on security topics introduced earlier in the course: the AWS Shared Responsibility Model, DDoS protection with AWS Shield, encryption key management with AWS KMS, and web application protection with AWS WAF.

Key Concepts

- **Six advantages** of cloud computing over on-premises infrastructure:
- **Expense advantage** – no upfront hardware/data center investment; pay only for what you consume.
- **Scale advantage** – AWS aggregates demand across customers to buy hardware/software at larger scale, lowering costs passed on to customers.
- **Capacity advantage** – stop guessing capacity; scale up/down on demand instead of over-provisioning.
- **Speed advantage** – new resources available in minutes (vs. 6–20 weeks on-premises), increasing agility.
- **Focus advantage** – teams focus on differentiating applications instead of maintaining infrastructure.
- **Global advantage** – deploy to multiple regions in minutes for lower latency and built-in DR/high-availability support.
- **Shared Responsibility Model**: AWS is responsible for security **"of the cloud"** (global infrastructure — Regions, Availability Zones, Edge Locations, and foundation services like compute/storage/database/networking); the customer is responsible for security **"in the cloud"** (guest OS, network/firewall configuration, platform/application/IAM, customer data, and encryption of data in transit/at rest).
- DDoS defense is layered: **AWS Shield Standard** is automatic/free baseline protection against common infrastructure-layer (**L3/L4**) attacks; **AWS Shield Advanced** adds expanded protection and access to the 24x7 **DDoS Response Team (DRT)** for application-layer (**L7**) attacks, typically paired with **AWS WAF** rules.
- **Envelope encryption** in **AWS KMS**: a two-tier key hierarchy where unique **data keys** encrypt the actual data, and a **master key** (which never leaves KMS) encrypts the data keys — limiting blast radius if one data key is compromised and making large-data encryption more efficient.
- KMS management capabilities: create keys with aliases, control which IAM users/roles can manage or use keys, automatic **annual key rotation**, disable/re-enable keys, and audit key usage via **AWS CloudTrail**.
- **WAF** operates at **Layer 7**, inspecting HTTP/HTTPS traffic against rules such as the **OWASP Top 10**, rate limiting (caps requests per user/session/IP), and whitelist/blacklist rules for more granular control than network ACLs.

AWS Services Covered

- **AWS Shield** – Managed DDoS protection; Standard tier is automatic and free, Advanced tier adds expanded protection and the DDoS Response Team for EC2, ELB, CloudFront, and Route 53.

- **AWS WAF** – Web application firewall; filters HTTP/HTTPS traffic using rules (OWASP Top 10, rate limiting, whitelist/blacklist) to mitigate application-layer (L7) attacks.
- **AWS Key Management Service (AWS KMS)** – Managed encryption key service; uses envelope encryption (master keys encrypt data keys) to protect data in S3, EBS, Redshift, and custom applications; integrates with CloudTrail for audit logging.
- **Amazon EC2** – Example resource protected by Shield Advanced and encrypted via KMS-managed keys.
- **Elastic Load Balancing** – Example resource protected by Shield Advanced.
- **Amazon CloudFront** – Example resource protected by Shield Advanced; also where most infrastructure-layer attacks are mitigated fastest (under 1 second).
- **Amazon Route 53** – Example resource protected by Shield Advanced (hosted zones).
- **Amazon S3** – Example data store encrypted using AWS KMS-managed data keys.
- **Amazon Redshift** – Example data warehouse cluster encrypted using AWS KMS-managed data keys.
- **AWS CloudTrail** – Used to audit AWS KMS key usage via logs.
- **AWS IAM** – Controls which users/roles can manage or use KMS keys.

Exam/Interview Tips

- Know the **Shared Responsibility** split cold: AWS secures **"of the cloud"** (physical infrastructure, AZs, regions, foundation services); the customer secures **"in the cloud"** (OS, network/firewall config, IAM, data encryption) — a near-guaranteed exam topic.
- **AWS Shield Standard** is automatic and free for all customers; **AWS Shield Advanced** is the paid tier that adds the DDoS Response Team and broader coverage — don't confuse the two tiers.
- **KMS** envelope encryption pattern: **master key** never leaves KMS and encrypts **data keys**; data keys (not the master key) directly encrypt customer data — this limits the damage if a single data key is exposed.
- **WAF** operates at **Layer 7** (HTTP/HTTPS) and is rule-based (OWASP Top 10, rate limiting, allow/deny lists) — distinguish it from **Shield**, which handles **L3/L4** infrastructure-layer DDoS attacks.
- Recall the **six cloud economics advantages** (expense, scale, capacity, speed, focus, global) as a quick framework for "why move to the cloud?" questions.
- KMS enforces **automatic annual rotation** only for AWS-managed and customer-managed keys with rotation enabled — imported key material must be rotated manually, which is a common exam distractor. *(editor's note)*

Cheat sheet generated for student reference from the Architecting on AWS v6.8 course materials. Not an official AWS publication.